

The Scouting Suite

Analisi e visualizzazione avanzata di statistiche calcistiche

Brivio Andrea — Matricola: 1089359

Fischetti Alessandro — Matricola: 1080491

Pesenti Lorenzo — Matricola: 1072635

Corso di Laurea in Ingegneria Informatica
Anno Accademico 2025/2026

Ingegneria del Software



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO



OBIETTIVO DEL PROGETTO

Realizzare un sistema per lo **scouting calcistico**, adatto ad ogni categoria



Analisi avanzata

- Più di 100 metriche statistiche



Filtraggio dinamico

- Combinabili con statistiche complesse



Visualizzazione intuitiva

- Dati di performance



Supporto decisionale

- Per scout e analisti



VALORE AGGIUNTO:

L'applicazione unisce il **rigore statistico** di dati complessi a un'**interfaccia utente semplice** e accessibile per professionisti dello sport.

DIFFICOLTÀ INCONTRATE

Le sfide superate

Gestione dataset complessi

- 100+ attributi per giocatore con relazioni complesse

Implementazioni query dinamiche

- Filtri combinabili in tempo reale

Integrazione tecnologie

- Spring Boot + Vaadin + JPA + H2

Design UI

- Bilanciare completezza e usabilità

Testing

- Dipendenze multiple e stati complessi

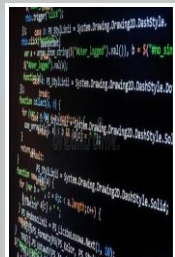
Coordinazione UML

- Allineamento diagrammi con implementazione

IMPLEMENTAZIONE - TECNOLOGIE E TOOL

Per l'implementazione è stato usato l'ambiente di sviluppo **Eclipse**.
Il Sistema è stato realizzato come **Progetto Maven** per la gestione delle dipendenze.

Le tecnologie utilizzate sono:



Backend

- Java 17, Spring Boot 3.1, Spring Data JPA



Frontend

- Vaadin Flow 24 (Java-based UI framework)



Database

- H2 Database (In-Memory per alta velocità)



Tools

- Maven, Lombok, Tablesaw (CSV parsing)



Testing

- JUnit 5, Mockito, Spring Boot Test



Quality

- SonarQube, PMD, JDepend

SOFTWARE CONFIGURATION MANAGEMENT



GitHub

Issue

Per assegnare e gestire attività

Push

Per caricare aggiornamenti

Pull

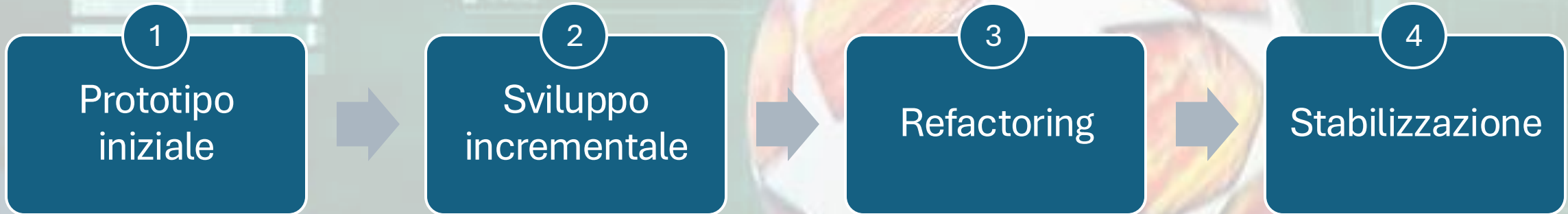
Per scaricare aggiornamenti

Commit

Per registrare modifiche

SOFTWARE LIFE CYCLE

Prototipazione Evolutiva + Sviluppo Incrementale



MOTIVAZIONE DELLA SCELTA:

- **Elevata incertezza iniziale** dovuta alla mancanza di esperienza;
- Necessità di **feedback rapido** su interfaccia utente;
- **Complessità dati** richiede approccio iterativo;
- **Flessibilità** nell'adattare requisiti emergenti.

ARCHITETTURA DEL SISTEMA

Design Pattern MVC
(model-view-
controller)

UI(vaadin) logica di business e dati

Strategy Pattern

Gestione filtri su dati eterogenei

Repository Pattern

Astrazione dell'accesso ai dati con Spring Data JPA

ETL Pipeline

Gestisce importazione, pulizia e normalizzazione

DESIGN PATTERN

Factory Method

- **PlayerSpecificationFactory** - Creazione dinamica di oggetti Specification per query complesse

Specification

- Pattern per costruire query complesse con criteri combinabili in AND/OR

DTO

- **PlayerFilterRequest** - Data Transfer Object per trasferimento dati tra layer

Strategy

- **StatQueryStrategy** - Algoritmi intercambiabili per diversi tipi di filtro statistico

Repository

- **PlayerRepository** - Astrazione dell'accesso ai dati, separazione logica/persistenza

Component

- **BrowserLauncher, FilterRowComponent** - Componenti riutilizzabili e testabili

MODELLAZIONE UML

Use Case Diagram - Requisiti funzionali e attori

- È stato definito per primo come risultato della fase dei requisiti.

Class Diagram – Struttura classi e relazioni

- Usato come modello per un'attività di model driven architecture, costruendo lo scheletro con il tool di generazione automatica del codice di Papyrus.
- Durante la ristrutturazione è stato ampiamente modificato per migliorare la modularità del codice.

State Machine Diagram - Stati del sistema

- Definitivo in fase iniziale per chiarire il flusso dell'interazione.
- È stato utilizzato per guidare lo sviluppo.

I seguenti diagrammi sono stati eseguiti successivamente allo sviluppo e sono serviti a esplicitare graficamente la struttura del sistema.

Activity Diagram - Flussi di processo

Sequence Diagram - Interazioni temporali

Communication Diagram - Collaborazione oggetti

Package Diagram - Organizzazione moduli

Component Diagram - Architettura componenti

APPROCCIO MODEL-DRIVEN DEVELOPMENT

Coerenza tra diagrammi UML e implementazione codice, documentazione completa, tracciabilità requisiti → design → codice

TESTING

Il sistema è stato validato attraverso un approccio ibrido:

TEST AUTOMATICO

- Test unitari (JUnit 5) per logica di business e filtri
- Test di integrazione per componenti Spring
- Analisi statica codice con SonarLint

VALIDAZIONE MANUALE

- Testing esplorativo dell'interfaccia con utenti target
- Verifica comportamento con dati reali e edge cases
- Validazione UX dei filtri complessi e colonne dinamiche

RISULTATI:

- Identificazione e **correzione di incoerenze** nella validazione input
- **Ottimizzazione query** per grandi dataset
- **Miglioramento dell'usabilità** dei filtri avanzati

MANUTENZIONE

Codice
modulare

Implementazione
Filtri

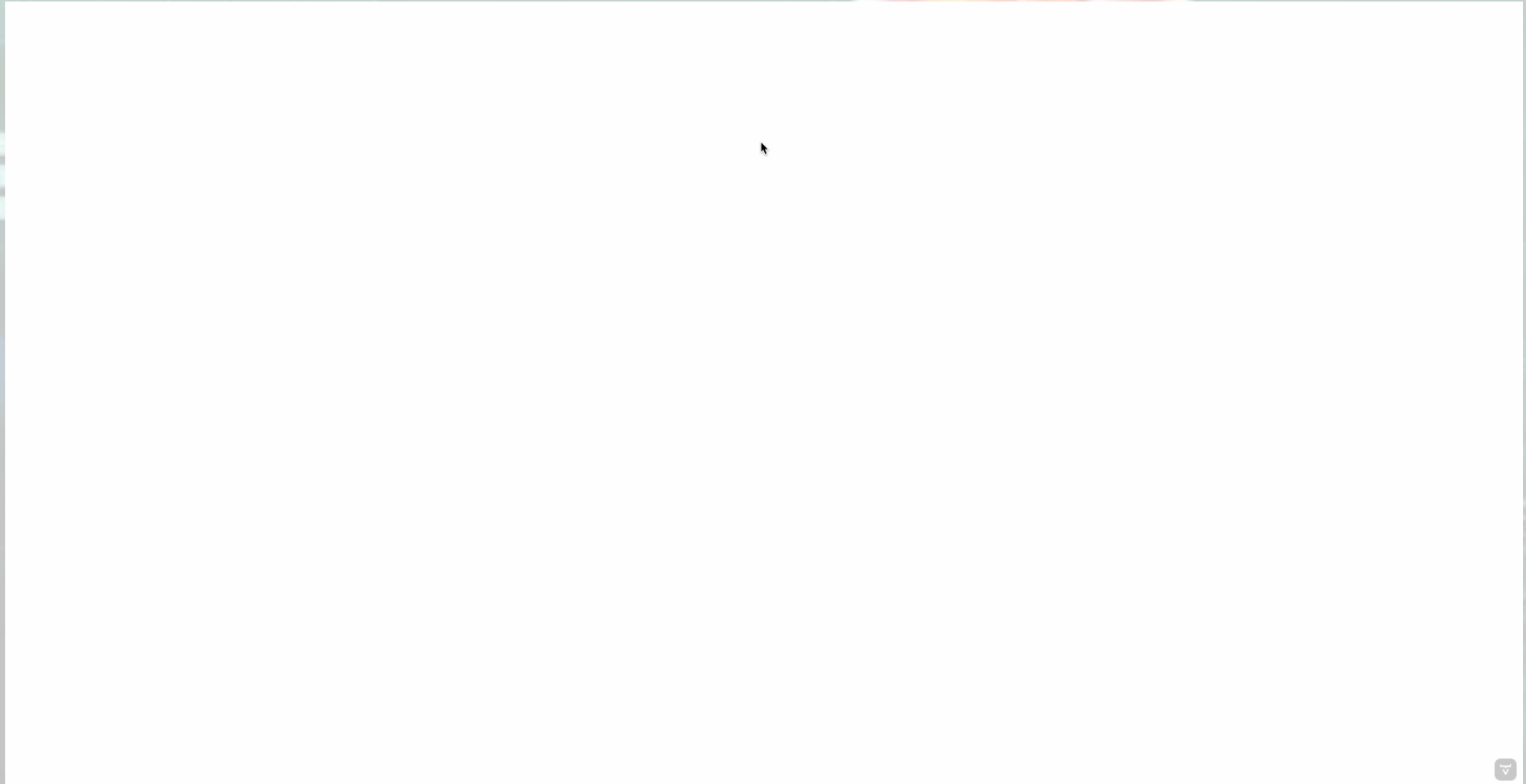
Git per gestione
modifiche

UML Papyrus

Organizzazione
e in Classi



DEMO PER MOSTRARE IL FLUSSO DELL'INTERAZIONE CON L'UTENTE



CONCLUSIONI

RISULTATI RAGGIUNTI

- **Sistema completo** per analisi calcistica avanzata
- **Architettura modulare**, estensibile e mantenibile
- **Pattern architetturali** ben applicati e documentati
- **Interfaccia utente intuitiva** per dati complessi
- Processo di **sviluppo strutturato** e documentato
- **Testing automatico** con alta copertura



The Scouting Suite

Grazie per l'attenzione

Brivio Andrea — Matricola: 1089359

a.brivio11@studenti.unibg.it

Fischetti Alessandro — Matricola: 1080491

a.fischetti@studenti.unibg.it

Pesenti Lorenzo — Matricola: 1072635

l.pesenti16@studenti.unibg.it



**UNIVERSITÀ
DEGLI STUDI
DI BERGAMO**

